

Fund Management System — Consolidated Workflow & Algorithm Reference

This document contains all four workflow artefacts as developed: Collection Cycle, Loan Lifecycle, Migration & Historical Cycle Resolution, and Reconciliation. Content is preserved as-is from the design session — tables, journal entries, algorithms, and state machines intact.

Contents

1. [Collection Cycle Workflow](#)
 2. [Loan Lifecycle Workflows](#)
 3. [Migration & Historical Cycle Resolution](#)
 4. [Reconciliation Workflow](#)
-
-

1. Collection Cycle Workflow

Overview

A five-phase end-to-end process from cycle open to settlement, covering contribution collection, late fee tiers, bank reconciliation, and reporting.

Account Structure

Account	Purpose
Master Bank Account	Cash from bank statement imports
Master Cash Account	Cash currently held by the system
Master Fund Account	Total value of the fund
Member Cash Account	Cash belonging to the member
Member Fund Account	Member's contributions to the fund

Master Account Invariant (assert nightly): - Master Fund Account = $\Sigma(\text{all Member Fund Accounts})$ - Master Cash Account = $\Sigma(\text{all Member Cash Accounts})$

Phase 1 — Cycle Initialisation (Day 1)

1. **Compute contribution amounts** — Calculate each member's contribution due for the cycle. Lock amounts into a pending contribution ledger (immutable snapshot — this is the basis for all late fee calculations).
 2. **Generate collection notices** — Notify all members of: amount due, due date, grace period end date, and each late fee tier threshold. Set status = **PENDING**.
 3. **Snapshot available balances** — Record each member's cash account balance at cycle open. Used to determine whether auto-debit can partially or fully satisfy the contribution.
-

Phase 2 — Collection Window (Days 1 – N)

Auto-debit — full balance available

If Member Cash Account balance ≥ contribution due, post immediately on Day 1 (or the configured debit date).

Journal entry:

	Account	Amount
DR	Member Cash Account	Contribution amount
CR	Master Fund Account	Contribution amount
CR	Member Fund Account	Contribution amount (memo)

Status → COLLECTED

Partial debit — insufficient balance

If balance < contribution due: debit available balance, mark remaining shortfall as **PARTIALLY PENDING**. Continue monitoring for incoming deposits.

Journal entry:

	Account	Amount
DR	Member Cash Account	Available balance
CR	Master Fund Account	Available balance
CR	Member Fund Account	Partial amount (memo)

Status → PARTIALLY PENDING

Real-time deposit matching

Any deposit arriving during the window (bank import or direct cash deposit) triggers an immediate re-evaluation. If the shortfall is covered, post the remainder and mark **COLLECTED**.

Design note: Every time a deposit posts to a member's cash account, the system should immediately check whether it closes an outstanding contribution shortfall — before the deposit becomes available for other uses. This prevents the gap where a member funds their account but the contribution remains flagged as overdue.

Phase 3 — Grace Period Close & Late Fee Engine (Day N+1 onward)

End-of-window sweep

At close of the collection window, flag all members with outstanding balances as **OVERDUE**. Record `overdue_since` timestamp — this is the immutable clock start for all late fee tier calculations.

The `days_overdue` counter must start from the exact close-of-business timestamp of the collection window, not the cycle start date. Disputes almost always center on this boundary.

Tiered late fee schedule

Days overdue	Status	Action
1 – 3	OVERDUE	Reminders only — no fee applied
After day 3	LATE — Tier 1	Apply Fee X — post to member cash account
After day 10	LATE — Tier 2	Apply Fee Y (replaces Tier 1) — post to member cash account
After day 20	LATE — Tier 3	Apply Fee Z (replaces Tier 2) — escalate, flag account

Fee model options:

- **Replacement model (standard):** Each new tier replaces the prior fee. Tier 2 reverses Tier 1 and posts Tier 2.
- **Cumulative model:** Each new tier is added on top of prior fees. Each tier posts an additional DR to the member's cash account.

Journal entry — late fee application:

	Account	Amount
DR	Member Cash Account	Late fee amount
CR	Late Fee Income Account	Late fee amount

Nightly batch algorithm

```
FOR each member WHERE status IN ('OVERDUE', 'LATE_T1', 'LATE_T2', 'LATE_T3'):
    days = today - overdue_since_date
    new_tier = lookup_tier(days)           // e.g. {3: T1, 10: T2, 20: T3}
    IF new_tier != member.current_tier:
        reverse_prior_fee_entry(member)    // if replacement model
        post_late_fee(member, new_tier.fee)
        update_status(member, new_tier.label)
        notify_member(member)
```

Phase 4 — Settlement & Bank Reconciliation (Ongoing)

Bank import path

Bank statement lines are imported and matched against pending transactions. On match, the Master Cash Account entry is cleared.

Journal entry:

	Account	Amount
DR	Master Bank Account	Deposit amount
CR	Master Cash Account	Deposit amount

Direct cash deposit path

Cash deposited directly is posted immediately to the member's Cash Account, then reconciled later with a matching bank import entry.

Journal entry:

	Account	Amount
DR	Master Cash Account	Deposit amount
CR	Member Cash Account	Deposit amount

Cycle close & carry-forward

At month-end, reconcile Master Bank / Master Cash / Master Fund balances. Any unresolved shortfalls carry into the next cycle with accumulated late fees and escalated status.

Phase 5 — Reporting & Audit Trail

Collection summary report

Per-member output: amount due, amount collected, outstanding balance, fees applied, current status. Exported for administrator review.

Immutable audit log

Every journal entry, status change, fee application, and deposit match is written to a tamper-evident log with timestamp and operator ID. Required for dispute resolution and regulatory compliance.

Member Status State Machine

PENDING

- balance covers → COLLECTED
- partial balance → PARTIALLY PENDING
 - deposit covers shortfall → COLLECTED
- window closes → OVERDUE
- window closes (zero balance) → OVERDUE
 - day 3+ → LATE Tier 1
 - day 10+ → LATE Tier 2
 - day 20+ → LATE Tier 3 (escalate)

Any OVERDUE / LATE state:

- payment received → SETTLING → COLLECTED

Configuration (per cycle)

Keep these in a configuration table — not hardcoded — to allow adjustments without deployment:

Parameter	Description
collection_window_days	Length of the collection window (N days)
tier_1_day_threshold	Days overdue before Tier 1 fee applies (e.g. 3)
tier_2_day_threshold	Days overdue before Tier 2 fee applies (e.g. 10)
tier_3_day_threshold	Days overdue before Tier 3 fee applies (e.g. 20)
fee_tier_1	Fee amount X
fee_tier_2	Fee amount Y
fee_tier_3	Fee amount Z
fee_model	replacement or cumulative

2. Loan Lifecycle Workflows

System account structure

Account	Purpose
Master Bank Account	Cash from bank statement imports
Master Cash Account	Cash currently held by the system
Master Fund Account	Total value of the fund
Member Cash Account	Cash belonging to the member
Member Fund Account	Member's contributions to the fund

Master account invariant (assert nightly): - Master Fund Account = $\Sigma(\text{all Member Fund Accounts})$ - Master Cash Account = $\Sigma(\text{all Member Cash Accounts})$

1. Loan eligibility & admin override

Eligibility gates (evaluated in sequence)

Gate 1 — Membership tenure Member must have at least X years of active membership, computed from join date to request date. > Fail → request rejected. Admin override can bypass this gate.

Gate 2 — Minimum fund balance Member's Fund Account balance must be ≥ Y amount at time of request. > Fail → request rejected. Admin override can bypass this gate.

Gate 3 — Delinquency check Member must have no active delinquency flag (missed EMIs, overdue contributions). Delinquent members are blocked from new loan requests. > Delinquent → request blocked. Admin override required to proceed.

Gate 4 — Active loan count Member may not exceed X active loans simultaneously (typically 1–2). A loan is active from first disbursement until full repayment. > Exceeds limit → request rejected. Admin override can bypass.

Gate 5 — Borrow limit Requested amount must be ≤ $X \times \text{Member Fund Account balance}$. Applies regardless of which fund tier the loan is drawn from. > Exceeds multiplier → request rejected or admin may adjust amount.

Gate 6 — Guarantor assignment Every loan request must name a guarantor — an active member in good standing. The guarantor may not be the same person as the borrower. Guarantor eligibility follows

the same delinquency rule. > No valid guarantor → request cannot be submitted. This gate cannot be overridden.

Admin override matrix

Gate	Admin can override?	Logged?
Tenure (X years)	Yes	Yes — mandatory reason required
Min fund balance (Y)	Yes	Yes — mandatory reason required
Delinquency flag	Yes	Yes — mandatory reason required
Active loan count	Yes	Yes — mandatory reason required
Borrow multiplier	Yes	Yes — mandatory reason required
No guarantor	No	N/A

2. Loan request submission & queue management

Submission flow

1. **Member submits loan request** Input: requested amount, loan type (standard or emergency), guarantor ID, grace period election (0, 1, or 2 cycles). System snapshots member's fund balance and active loan count at submission time.
 2. **Eligibility gates evaluated** All gates from Section 1 are evaluated in sequence. Any failure either rejects the request or raises an admin override flag. > If admin override was used: request enters the queue tagged OVERRIDE with override reason logged.
 3. **Loan tier assignment** System maps the requested amount to an EMI tier (see Section 3). The tier determines the EMI amount and the associated fund tier from which the loan will be drawn.
 4. **Queue insertion** Standard requests are appended to the FIFO queue ordered by submission timestamp. Emergency requests are inserted at the front of the queue ahead of all pending standard requests. Multiple emergencies among themselves follow FIFO order. > Queue is processed by admin in order. Admin may not skip positions except via the emergency priority mechanism.
 5. **Status → PENDING_ADMIN** Member and guarantor are notified of queue position. Request stays in PENDING_ADMIN until admin processes it.
-

EMI tier lookup (configurable)

Loan amount range	EMI
1K – 10K	1K
11K – 30K	1.5K
31K – 60K	2.5K
Emergency	Configurable per request

Queue insertion algorithm

```
FUNCTION insert_to_queue(request):  
  IF request.type == 'EMERGENCY':  
    position = front_of_queue()  
  ELSE:  
    position = end_of_queue()  
  queue.insert(request, position)  
  assign_queue_number(request)  
  notify(member, guarantor, position)  
  set_status(request, 'PENDING_ADMIN')
```

3. Fund tier structure & availability

Fund tiers define how much of the Master Fund Account is available to fund loans in each loan tier. Tiers are configurable from 0%–100% and may overlap.

Fund tier model

Fund tier	Linked loan tier	Allocation	Overlaps allowed?
Fund Tier A	Loan Tier 1 (1K–10K)	0–100% configurable	Yes
Fund Tier B	Loan Tier 2 (11K–30K)	0–100% configurable	Yes
Fund Tier C	Loan Tier 3 (31K–60K)	0–100% configurable	Yes
Fund Tier E (Emergency)	None — emergency requests only	0–100% configurable	Yes

Overlap means the same portion of the Master Fund Account may count toward multiple fund tiers. Availability is checked at disbursement time against the current Master Fund balance $\times$ tier percentage.

Loan allocation split

Each loan is funded from two sources, computed at approval time:

Portion	Definition
Member's portion	Member Fund Account balance (up to loan amount)
Master's portion	Loan amount – Member's portion
Repayment threshold	Master portion + X% of requested loan amount

The loan is considered **fully repaid** when the master fund portion plus the X% settlement threshold has been repaid. The member's own portion represents their equity in the fund and is excluded from the guarantor's liability.

Fund availability check algorithm

```
FUNCTION check_fund_availability(loan_tier, amount):  
  tier_pct = config.fund_tier[loan_tier].allocation_pct  
  tier_pool = master_fund_balance * tier_pct  
  committed = sum of all active disbursements against this tier
```

```

available = tier_pool - committed
RETURN available >= amount    // full approval
      OR available > 0        // partial disbursement possible

```

4. Approval, partial disbursement & activation

Approval flow

1. **Admin reviews next request in queue** Admin sees: member name, requested amount, loan tier, fund tier availability, queue position, override flags. Emergency requests are clearly marked.
2. **Fund tier availability evaluated** System checks available balance in the associated fund tier.
 - Sufficient funds → full approval available.
 - Partial funds → admin may disburse partial amount.
 - No funds → request deferred or rejected.
3. **Admin decision: approve / partial / reject** Admin records decision with optional notes. Partial approval requires specifying the disbursed amount. Rejection requires a reason.
4. **Disbursement posted (per tranche)** Each disbursement — whether partial or full — triggers the following journal entries immediately:

	Account	Amount
DR	Master Fund Account	Disbursed amount
CR	Member Cash Account	Disbursed amount
Memo	Member Fund Account	–Disbursed amount (reflected)

The member's cash account is credited. The member may then request a cash-out via a separate cash-out process flow.

5. **Loan activation check** The loan is NOT active until the total approved amount has been fully disbursed. Partial disbursements accumulate under status `PARTIALLY_DISBURSED`.
 - Fully disbursed → status = `ACTIVE`. Repayment schedule is built (see Section 5).
 - Partially disbursed → awaiting further tranches. No repayment schedule yet. Grace period clock has not started.
 6. **Member & guarantor notified** On full activation: notify of loan amount, repayment schedule start date (accounting for grace period election), EMI amount, and full repayment threshold.
-

5. Repayment schedule construction & grace period

Grace period election

The member elects their grace period preference at request submission time. This election is locked and cannot be changed after approval.

Election	Effect
0 cycles (no grace)	First EMI due in the immediate next collection cycle
1 cycle grace	First EMI due in cycle N+2 (one cycle skipped)
2 cycles grace	First EMI due in cycle N+3 (two cycles skipped)

Grace cycles are interest-free deferral — no EMI and no late fee during the grace window. The contribution exemption still applies during grace periods.

Schedule construction algorithm

```

FUNCTION build_schedule(loan):
    emi          = lookup_emi_tier(loan.amount)
    threshold    = loan.master_portion + (loan.amount * settlement_pct)
    grace_cycles = loan.grace_election          // 0, 1, or 2
    start_cycle  = current_cycle + 1 + grace_cycles
    cycle        = start_cycle
    total_repaid = 0
    schedule     = []

    WHILE total_repaid < threshold:
        schedule.append({cycle, emi, status: 'PENDING'})
        total_repaid += emi
        cycle        += 1

    // Last instalment adjusted to exact remainder
    schedule[-1].amount = threshold - (total_repaid - emi)
    loan.repayment_schedule = schedule
    loan.full_repayment_threshold = threshold
    RETURN schedule

```

Repayment collection

Each EMI cycle is collected via the exact same mechanism as monthly contributions: auto-debit from member cash account, partial debit if insufficient, real-time deposit matching, and the identical tiered late fee schedule.

Journal entry — EMI collection:

	Account	Amount
DR	Member Cash Account	EMI amount
CR	Master Fund Account	EMI amount
CR	Loan Repayment Ledger	EMI amount (memo)

6. Early settlement — full & partial

Full early settlement

1. Member requests full settlement. System computes outstanding threshold balance: `threshold - total_repaid_to_date`. Member must fund their cash account with this amount before settlement is posted.
2. Settlement journal entry:

	Account	Amount
DR	Member Cash Account	Remaining threshold
CR	Master Fund Account	Remaining threshold

3. Loan status → REPAID. Contribution exemption lifted next cycle. Guarantor liability released.
-

Partial early settlement

1. **Member specifies partial amount.** Amount must be 1 full EMI. System validates that the remaining balance after payment still meets the minimum threshold trajectory.

2. **Member elects schedule treatment:**

Option A — Roll-up (compress): Partial payment is applied forward. Upcoming EMI cycles that are fully covered by the payment are marked as settled. The remaining schedule shortens (fewer cycles remain).

Option B — Skip cycles: Partial payment covers X upcoming cycles which are skipped. The original schedule length is preserved — the member has X cycles of relief before EMIs resume.

3. **Schedule rebuilt.** System rebuilds the repayment schedule from the current cycle forward applying the chosen option. Member and guarantor are notified of the updated schedule.

	Account	Amount
DR	Member Cash Account	Partial amount
CR	Master Fund Account	Partial amount

Partial settlement schedule rebuild algorithm

```

FUNCTION apply_partial_settlement(loan, amount, option):
    remaining_threshold = loan.threshold - loan.total_repaid
    cycles_covered = floor(amount / loan.emi)

    IF option == 'ROLLUP':
        mark next cycles_covered pending EMIs as SETTLED
        rebuild remaining schedule from (current + cycles_covered + 1)

    IF option == 'SKIP':
        mark next cycles_covered pending EMIs as SKIPPED
        resume EMIs from (current + cycles_covered + 1)
        // total cycle count stays the same; gaps are inserted

    loan.total_repaid += amount
    IF loan.total_repaid >= loan.threshold:
        close_loan(loan)

```

7. Guarantor liability & delinquency escalation

Missed EMI escalation ladder

Missed EMIs	Action	Borrower status
1 – (X–1)	Late fees apply per collection cycle rules	Overdue
X missed	Guarantor notified — formally warned of upcoming liability	Delinquent
Y missed	Guarantor assumes full liability; loan transferred	Suspended

Guarantor liability scope

The guarantor is responsible **only for the master fund portion** of the remaining repayment — the portion originally drawn from the Master Fund Account, plus the settlement threshold percentage. The member's own fund equity portion is excluded.

Component	Definition
Remaining master portion	(Master portion of original loan) – (repayments already credited to master portion)
+ Settlement threshold	X% of original requested loan amount
= Guarantor obligation	Total amount guarantor must repay

Loan transfer flow

1. **Automatic loan transfer at Y missed EMIs.** System re-assigns loan ownership to the guarantor. The guarantor becomes the borrower of record for the remaining obligation.
2. **Borrower suspended.** Original borrower's account is flagged SUSPENDED. Suspended members cannot request new loans, make withdrawals, or submit transactions requiring admin approval until the loan is fully resolved and admin lifts the suspension. > A suspended member cannot be a guarantor on any other loan.
3. **Guarantor repayment schedule rebuilt.** A new schedule is built for the guarantor based on the remaining obligation. The guarantor's active loan count is incremented (counts against their active loan limit). Late fees apply to the guarantor's repayments under the same collection cycle rules.
4. **Resolution & reinstatement.** Once the transferred loan is fully repaid by the guarantor, admin may review the original borrower's suspension. Reinstatement is not automatic — it requires an explicit admin action.

Escalation algorithm

```
NIGHTLY_JOB: evaluate_missed_emis()
FOR each active loan:
    missed = count EMIs with status MISSED or OVERDUE

    IF missed == X:
        notify(guarantor, 'liability_warning')
```

```

set_borrower_status('DELINQUENT')

IF missed >= Y:
    guarantor_obligation = calc_guarantor_liability(loan)
    transfer_loan(loan, new_owner=guarantor)
    suspend_member(loan.original_borrower)
    rebuild_schedule(guarantor, guarantor_obligation)
    notify(guarantor, 'liability_active')
    notify(borrower, 'account_suspended')

```

8. Contribution exemption during loan repayment

A member under an active loan repayment schedule is exempt from making standard monthly contributions to the fund. This prevents double-charging members who are already servicing a loan.

Exemption rules

Condition	Contribution required?	Notes
Loan ACTIVE, in repayment	Exempt	No contribution debit for this cycle
Loan in grace period (pre-repayment)	Exempt	Grace cycles also exempt
Loan PARTIALLY_DISBURSED	Exempt	Exempt from activation date onward
Loan REPAID this cycle	Due	Exemption lifts; contribution due next cycle
Loan transferred to guarantor — original borrower	Due	Borrower is suspended but contributions resume
Loan transferred to guarantor — guarantor	Exempt	Guarantor is now in repayment; standard exemption applies
Multiple active loans	Exempt	One active loan is sufficient for full exemption

Collection cycle integration algorithm

```

MONTHLY_COLLECTION_JOB:
    FOR each member:
        IF has_active_loan(member) OR in_grace_period(member):
            skip_contribution_debit(member)
            log('EXEMPT - active loan', member, cycle)
        ELSE:
            proceed_with_contribution_collection(member)

```

Fund account continuity note

While exempt from contributions, the member's Fund Account balance does not grow during the loan period. This directly affects future loan eligibility, since the borrow limit is computed as $X \times \text{Fund Account balance}$. Members should be informed that loan repayment periods pause their fund accumulation.

Appendix — Configurable parameters

All of the following should be stored in a configuration table, not hardcoded, to allow cycle-by-cycle adjustment without deployment.

Parameter	Description
<code>min_membership_years</code>	Minimum years of membership before a loan can be requested (X)
<code>min_fund_balance</code>	Minimum fund account balance required to request a loan (Y)
<code>borrow_multiplier</code>	Maximum loan amount as a multiple of fund balance (X×)
<code>max_active_loans</code>	Maximum number of simultaneous active loans per member
<code>settlement_threshold_pct</code>	% of requested loan added to master portion to define full repayment threshold
<code>grace_period_options</code>	Allowed grace period elections (e.g. 0, 1, 2 cycles)
<code>emi_tiers</code>	Table of loan amount ranges and their associated EMI amounts
<code>fund_tier_allocations</code>	Per-tier % of master fund available for each loan tier
<code>late_fee_tier_days</code>	Day thresholds for Tier 1 / 2 / 3 late fees (e.g. 3, 10, 20)
<code>late_fee_amounts</code>	Fee amounts X, Y, Z for each late fee tier
<code>late_fee_model</code>	replacement or cumulative
<code>guarantor_warning_threshold</code>	Missed EMIs before guarantor notification (X)
<code>guarantor_liability_threshold</code>	Missed EMIs before automatic loan transfer (Y)
<code>collection_window_days</code>	Length of each monthly collection window

Appendix — Loan status state machine

```
PENDING_ADMIN
  rejected → REJECTED (terminal)
  approved (partial) → PARTIALLY_DISBURSED
    fully disbursed → ACTIVE
      (grace period) → ACTIVE [no EMI due yet]
      in repayment → ACTIVE
        early full settlement → REPAID (terminal)
        early partial settlement → ACTIVE [schedule rebuilt]
        X missed EMIs → DELINQUENT [guarantor warned]
        Y missed EMIs → TRANSFERRED [guarantor owns loan]
          guarantor repays fully → REPAID (terminal)
      all EMIs collected → REPAID (terminal)
```

Appendix — Journal entry summary

Event	DR	CR
Loan disbursement	Master Fund Account	Member Cash Account
Disbursement memo	Member Fund Account (—)	—
EMI collection	Member Cash Account	Master Fund Account

Event	DR	CR
Late fee — loan EMI	Member Cash Account	Late Fee Income Account
Full early settlement	Member Cash Account	Master Fund Account
Partial early settlement	Member Cash Account	Master Fund Account
Monthly contribution (non-exempt)	Member Cash Account	Master Fund Account
Late fee — contribution	Member Cash Account	Late Fee Income Account
Bank import clearing	Master Bank Account	Master Cash Account
Direct cash deposit	Master Cash Account	Member Cash Account

3. Migration & Historical Cycle Resolution

Overview

Members migrated from a legacy system may have a join date extending far into the past with no contribution data available. The new system establishes opening balances and resolves all historical cycles without triggering delinquency rules prematurely.

Phase 1 — Member onboarding

Step 1 · Collect migration data per member

From the old system or manual records, gather: join date, cash balance at migration date, fund balance at migration date, and any known history flags. If no historical data exists, balances default to zero.

Step 2 · Post opening balance — Member Cash Account

	Account	Amount
DR	Master Cash Account	Cash opening balance
CR	Member Cash Account	Cash opening balance

Step 3 · Post opening balance — Member Fund Account

	Account	Amount
DR	Master Fund Account	Fund opening balance
CR	Member Fund Account	Fund opening balance

Both entries are tagged `MIGRATION_OPENING_BALANCE` with the migration **effective date** (not the posting date). This distinguishes them from regular contributions in all reporting.

Step 4 · Set the migration cutoff date

Record `migration_cutoff_date` — the date from which the new system takes over. The member's join date and the cutoff together define the historical window.

The cutoff date is immutable once set. Changing it invalidates all cycle resolution records. Admin approval required to amend.

Step 5 • Generate historical cycle stubs

System auto-generates one cycle record per month between `join_date` and `migration_cutoff_date`. Each stub is created with status `UNRESOLVED` — **not** `MISSED`.

Do not mark historical cycle stubs as MISSED at creation. Doing so would trigger the late fee engine and delinquency flags against the member before any resolution process has run.

Step 6 • Member profile status → `MIGRATION_PENDING`

While in this status: the late fee engine is suppressed, delinquency rules do not apply, and guarantor notifications cannot fire. Status lifts to `ACTIVE` only after all historical cycles are resolved and admin grants clearance.

Phase 2 — Historical cycle classification

Admin classifies each stub individually or in batch by date range.

Classification	Meaning	System action
<code>WAIVED</code>	Predates available records; admin grants full waiver	Closed — zero obligation, audit entry only
<code>BACKDATED_PAID</code>	Evidence exists of payment in old system	Marked settled — no additional debit
<code>BACKDATED_DUE</code>	Cycle was genuinely missed; obligation exists	Converted to payable — resolution method required
<code>OB_ABSORBED</code>	Opening fund balance already accounts for these contributions	Batch-closed — no debit, opening balance flagged as settlement
<code>ESCALATED</code>	Disputed or requires further investigation	Frozen — does not count toward delinquency while in this state

Batch vs individual classification

Batch classification (recommended for long histories): Admin sets a policy rule per member or cohort — e.g. “all cycles before year Y are `WAIVED`; cycles from year Y onward require individual review.” System applies the rule across all matching stubs in one operation.

Individual classification: Admin reviews each cycle stub one by one. Suitable when partial records exist in the old system and specific cycles can be matched to payment evidence from legacy data.

Opening balance absorption — when to use it

If the migrated fund opening balance was computed to already represent cumulative contributions, historical cycles should be classified `OB_ABSORBED` in bulk. This avoids double-counting — the opening balance entry already credits what those cycles would have contributed.

Never use `OB_ABSORBED` if the opening balance was an estimate or approximation. Use `WAIVED` instead, and record the basis of the opening balance in the audit entry.

Phase 3 — Resolution methods for `BACKDATED_DUE` cycles

Method A — Lump-sum settlement

Member pays all outstanding backdated cycles in a single transaction. All affected cycles are marked `SETTLED` simultaneously.

	Account	Amount
DR	Member Cash Account	Total backdated amount
CR	Master Fund Account	Total backdated amount
CR	Member Fund Account	Total backdated amount (memo)

Fastest path to clearing MIGRATION_PENDING. Recommended when the member has sufficient cash account balance.

Method B — Instalment plan (spread over future cycles)

Total backdated amount divided into equal instalments over N future cycles, running alongside regular contributions. Each instalment uses the same collection mechanism and late fee rules as a standard EMI if missed.

	Account	Amount
DR	Member Cash Account	Instalment amount
CR	Master Fund Account	Instalment amount
CR	Member Fund Account	Instalment amount (memo)

The member is NOT automatically exempt from regular contributions during an instalment plan. Both obligations run in parallel unless admin explicitly grants a concurrent exemption.

Method C — Opening balance offset

Applicable only when opening fund balance = total backdated obligations. Admin offsets historical cycles directly against the opening balance. No cash movement required. Member consent must be recorded before posting.

	Account	Amount
DR	Member Fund Account	Total backdated amount
CR	Master Fund Account	Total backdated amount

Hybrid — partial waiver + partial settlement

Admin may apply different classifications and methods to different date ranges within a single member's history. Each range is handled independently and produces separate audit records.

Late fee treatment for backdated cycles

Late fees are **never** automatically applied to BACKDATED_DUE cycles, regardless of how long they have been outstanding. The late fee engine is suppressed for all cycles tagged with origin MIGRATION. Manual admin override required if fees are deemed appropriate — override reason must be logged.

Phase 4 — Delinquency clearance & ACTIVE status reinstatement

Clearance conditions (all must be met)

Condition	Required?
No UNRESOLVED stubs remain	Mandatory

Condition	Required?
All BACKDATED_DUE cycles have a resolution method assigned	Mandatory
No ESCALATED cycles remain open	Mandatory
Lump-sum or offset fully posted (if Method A or C chosen)	Conditional
Instalment schedule built and first date set (if Method B chosen)	Conditional
Admin sign-off	Mandatory

Clearance flow

1. System validates all mandatory conditions. Any unmet condition blocks the clearance action and surfaces a checklist.
2. Admin confirms resolution summary (cycles waived, settled, on instalment, offset applied) and submits approval.
3. Member status → **ACTIVE**. Normal operating rules apply from the next collection period. If an instalment plan is active, it runs in parallel with regular contributions.
4. Member receives migration clearance summary: opening balances confirmed, historical cycles resolved by type, any ongoing instalment obligations, and the date from which normal rules apply.

Partial clearance (advanced)

For members with very long histories, admin may grant **PARTIAL_CLEARANCE_GRANTED: ACTIVE** status is issued for all new-cycle operations while a subset of **ESCALATED** cycles continues to be investigated in the background. The escalated cycles are ring-fenced and cannot affect the member's operational status.

Journal entry tag reference

Tag	Event
MIGRATION_OPENING	Opening balance (cash or fund)
MIGRATION_WAIVER	Cycle waiver — zero entry, audit log only
MIGRATION_BACKDATED_PAID	Cycle marked settled — no cash movement
MIGRATION_OB_ABSORBED	Cycle absorbed into opening balance — no cash movement
MIGRATION_LUMPSUM	Lump-sum backdated settlement
MIGRATION_INSTALMENT	Individual instalment collected
MIGRATION_OB_OFFSET	Opening balance offset against backdated obligation
MIGRATION_MANUAL_FEE	Manual late fee override (admin only)

All migration journal entries carry both **effective_date** (the historical month) and **posting_date** (actual date of entry). This allows historical contribution reports to show correct period totals.

Algorithms

1 · Historical cycle stub generation

```
FUNCTION generate_historical_stubs(member):
```

```

cycle = first_cycle_month(member.join_date)
stubs = []
WHILE cycle <= member.migration_cutoff_date:
    stubs.append({
        member_id:      member.id,
        cycle_date:      cycle,
        status:          'UNRESOLVED',    // NOT 'MISSED'
        origin:          'MIGRATION',
        amount_due:      lookup_contribution_rate(cycle),
        late_fee_exempt: true              // permanently suppressed
    })
    cycle = next_month(cycle)
member.status = 'MIGRATION_PENDING'
RETURN stubs

```

2 · Batch classification

```

FUNCTION batch_classify(member, date_range, classification, method=null):
    stubs = get_stubs(member, status='UNRESOLVED', dates=date_range)
    FOR stub IN stubs:
        stub.classification = classification
        stub.classified_by  = admin.id
        stub.classified_at  = now()
        IF classification == 'BACKDATED_DUE':
            stub.resolution_method = method
            stub.status = 'PENDING_RESOLUTION'
        ELSE:
            stub.status = 'CLOSED'
        log_audit(stub)
    IF method == 'B':
        build_instalment_schedule(member, stubs)

```

3 · Instalment schedule builder

```

FUNCTION build_instalment_schedule(member, due_stubs):
    total_due = sum(s.amount_due FOR s IN due_stubs)
    n         = admin_config.migration_instalment_cycles
    instalment = ceil(total_due / n)
    start_cycle = next_active_cycle()
    schedule = []
    FOR i IN range(n):
        amt = instalment IF i < n-1
            ELSE total_due - (instalment * (n-1))
        schedule.append({
            cycle: start_cycle + i,
            amount: amt,
            status: 'PENDING',
            type:   'MIGRATION_INSTALMENT'
        })
    member.migration_instalment_schedule = schedule
    notify(member, schedule)

```

4 · Clearance eligibility check

```

FUNCTION check_clearance_eligible(member):

```

```

IF any stub WHERE status == 'UNRESOLVED':
    RETURN FAIL, 'Unresolved stubs remain'
IF any stub WHERE status == 'ESCALATED':
    RETURN FAIL, 'Escalated cycles not resolved'
IF any stub WHERE classification == 'BACKDATED_DUE'
    AND resolution_method IS NULL:
    RETURN FAIL, 'No resolution method assigned'
IF method == 'A' AND lump_sum_not_posted:
    RETURN FAIL, 'Lump-sum not posted'
IF method == 'B' AND schedule_not_built:
    RETURN FAIL, 'Instalment schedule missing'
IF method == 'C' AND offset_not_posted:
    RETURN FAIL, 'Opening balance offset not posted'
RETURN PASS

FUNCTION grant_clearance(member, admin):
    IF check_clearance_eligible(member) != PASS: RAISE error
    member.status = 'ACTIVE'
    member.migration_cleared_at = now()
    member.migration_cleared_by = admin.id
    log_audit('MIGRATION_CLEARANCE_GRANTED', member, admin)
    notify(member, 'migration_complete')

```

Master account reconciliation — migration integrity check

```

ASSERT: Master Fund Account balance
    == Σ(all Member Fund Account balances)
    + Σ(all BACKDATED_DUE obligations not yet collected)

ASSERT: Master Cash Account balance
    == Σ(all Member Cash Account balances)

IF either assertion fails:
    flag MIGRATION_RECONCILIATION_ERROR
    block clearance for all affected members
    require admin investigation before proceeding

```

4. Reconciliation Workflow

Overview

The reconciliation control layer sits above all operational processes as a continuous validation mechanism. It does not replace any workflow — it validates, detects drift, and routes discrepancies to either automatic resolution or a human queue.

Two execution modes: - Nightly batch — full sweep of all domains at a fixed time before any operational posting begins - Real-time event trigger — fires on every posted transaction; validates double-entry integrity and exemption rules immediately

Master account invariant (foundation check — must pass before all other checks):

Master Fund Account balance = $\Sigma(\text{all Member Fund Account balances})$
+ $\Sigma(\text{all BACKDATED_DUE obligations not yet collected})$

Master Cash Account balance = $\Sigma(\text{all Member Cash Account balances})$

Discrepancy classification

Every detected discrepancy is classified before routing:

Type	Description
Timing difference	Entry exists on one side; counterpart expected within N days
Amount mismatch	Entry exists on both sides but amounts differ
Missing entry	One side of a journal entry is absent
Duplicate entry	Same transaction posted more than once
Status mismatch	Account or cycle status inconsistent with posted entries

Auto-resolve vs manual routing

Condition	Route
Delta configured tolerance	Auto-resolve as rounding adjustment
Delta matches a known in-flight transaction	Defer 24h (TIMING_DIFFERENCE); escalate after 48h
Delta exceeds tolerance, no in-flight match	Manual exception queue
Auto-resolve attempted but failed	Manual exception queue (severity = ESCALATED)

Domain 1 — Master account reconciliation

Run first in every nightly batch. A MASTER_IMBALANCE_UNRESOLVED halts all further posting.

Check sequence

1. Snapshot all account balances at a fixed point-in-time (e.g. 00:01 daily) before any batch jobs run.
2. Assert both invariant equations.
3. If delta tolerance $\rightarrow$ auto-resolve as rounding adjustment.
4. If delta matches in-flight transaction $\rightarrow$ defer 24h, re-check. Escalate after 48h.
5. If delta unexplained $\rightarrow$ raise MASTER_IMBALANCE_UNRESOLVED (CRITICAL). Halt all posting.

No contribution, EMI, fee, or disbursement posting should occur while a MASTER_IMBALANCE_UNRESOLVED is open. Posting against an imbalanced master account compounds the error.

Rounding adjustment journal entry

Account	Amount
DR Reconciliation Suspense Account	Delta amount

	Account	Amount
CR	Master Fund / Cash Account	Delta amount

Tag: RECON_AUTO_ROUNDING

Domain 2 — Contribution reconciliation

Per-member, per-cycle validation. Run at end of each collection window and again 48h later.

Check	Auto-resolve?	Manual trigger?
Member debited but Master Fund not credited	Yes — post missing CR leg	If member fund memo also missing
Master Fund credited but member not debited	No	Always — orphan credit
Amount collected amount due (within tolerance)	Yes	If delta > tolerance
Cycle marked COLLECTED but debit not posted	No	Always — status mismatch
Cycle marked PENDING past window close date	Yes — re-run debit attempt	If debit attempt fails again
Duplicate debit for same cycle	No	Always — reverse one, verify
MIGRATION_PENDING	Yes — reverse the debit	If reversal fails
member debited for live cycle		
Contribution collected from loan-exempt member	Yes — reverse collection	Always notify member

Auto-resolve: missing credit leg

	Account	Amount
DR	Member Cash Account	(existing — already posted)
CR	Master Fund Account	Contribution amount
CR	Member Fund Account (memo)	Contribution amount

Tag: RECON_AUTO_CONTRIBUTION_CR_FIX

Exempt member reversal journal

	Account	Amount
CR	Member Cash Account	Contribution refunded
DR	Master Fund Account	Contribution refunded

Tag: RECON_EXEMPT_REVERSAL

Domain 3 — EMI & loan disbursement reconciliation

EMI checks

Check	Auto-resolve?	Manual trigger?
EMI collected but loan repayment ledger not updated	Yes — post memo entry	If threshold recalculation affected
EMI marked MISSED but cash account had sufficient funds	No	Always
Total collected exceeds loan repayment threshold	Yes — refund overpayment	If over-collection > 1 EMI
Loan status ACTIVE but all EMIs marked COLLECTED	Yes — close loan, update status	Never (automatic)
Guarantor and borrower both debited same cycle	No	Always — duplicate collection
Grace cycle has an EMI debit posted	Yes — reverse debit	If reversal produces imbalance

Disbursement checks

Check	Auto-resolve?	Manual trigger?
Master Fund debited but member cash not credited	No	Always — incomplete disbursement
Member Fund memo not reflected on disbursement	Yes — post memo correction	If affects loan allocation calculation
Loan ACTIVE but disbursed amount < approved amount	No	Always — premature activation
Fund tier over-committed (disbursements exceed tier pool)	No	Always — requires admin correction
Repayment schedule built before full disbursement	Yes — void schedule, rebuild after full disburse	If any EMIs already collected

EMI over-collection refund journal

	Account	Amount
CR	Member Cash Account	Overpaid amount
DR	Master Fund Account	Overpaid amount

Tag: RECON_EMI_OVERPAYMENT_REFUND

Domain 4 — Bank clearing reconciliation

Automated matching algorithm

```
FOR each bank_statement_line imported today:
    candidates = find_cash_account_entries(
        amount      = bank_line.amount ± tolerance,
        date_range  = bank_line.date ± 3 days,
        status      = 'PENDING_CLEARANCE'
```

```

)
IF candidates.count == 1:
    auto_match(bank_line, candidates[0])
    post_clearing_entry(bank_line, candidates[0])
    status = 'CLEARED'
ELIF candidates.count > 1:
    raise RECON_AMBIGUOUS_MATCH → manual queue
ELIF candidates.count == 0:
    raise RECON_UNMATCHED_BANK_LINE → manual queue

```

Auto-matched clearing journal

	Account	Amount
DR	Master Bank Account	Deposit amount
CR	Master Cash Account	Deposit amount

Tag: RECON_AUTO_BANK_CLEAR

Bank clearing exception types

Exception	Likely cause	Admin action
UNMATCHED_BANK_LINE	Unknown depositor or wrong account	Identify depositor, post to correct member account, clear
UNMATCHED_CASH_ENTRY	System entry has no bank line — voided or wrong amount	Verify with bank, void or adjust cash entry
AMBIGUOUS_MATCH	Multiple pending entries with same amount and date	Use reference number or depositor ID to disambiguate
AMOUNT_MISMATCH	Bank amount differs (fee deduction, FX)	Post adjustment entry for the difference, clear original
STALE_PENDING	Cash entry pending > 30 days with no bank match	Investigate; void or escalate for bank trace

Direct cash deposits unmatched by a bank line within N days raise CASH_DEPOSIT_UNBANKED.

Domain 5 — Late fee reconciliation

Check	Auto-resolve?	Manual trigger?
Fee applied at wrong tier (day count mismatch)	Yes — reverse and repost correct tier	If member disputes and timing is boundary-case
Fee applied to migration cycle (origin = MIGRATION)	Yes — always auto-reversed	Never
Fee applied to loan-exempt member contribution cycle	Yes — reverse fee	If exemption status is disputed
Fee applied to grace-period EMI cycle	Yes — always auto-reversed	Never
Fee posted to wrong account (fee income vs master fund)	No	Always — reroute entry

Check	Auto-resolve?	Manual trigger?
Replacement model: prior tier not reversed when new tier applied	Yes — reverse prior, repost current	If prior tier already collected in cash
Fee Income Account balance Σ (all posted late fees)	No	Always — investigate orphaned entries

Fee tier correction journal (replacement model)

	Account	Amount
CR	Member Cash Account	Wrong tier fee (reverse)
DR	Late Fee Income Account	Wrong tier fee (reverse)
DR	Member Cash Account	Correct tier fee
CR	Late Fee Income Account	Correct tier fee

Tag: RECON_AUTO_FEE_TIER_CORRECTION

Fee income integrity assertion

ASSERT: Late Fee Income Account balance
 == Σ (all posted late fee journal entries, net of reversals)

IF assertion fails:
 compute difference by scanning individual fee entries
 identify orphaned or duplicate entries
 raise RECON_FEE_INCOME_MISMATCH → manual queue

Domain 6 — Migration reconciliation

Opening balance integrity checks

Check	Auto-resolve?	Manual trigger?
Opening balance entry missing one leg	No	Always — incomplete journal
Σ (MIGRATION_OPENING entries) master account snapshot at migration date	No	Always — batch onboarding error
Member has opening entry but no cutoff date set	Partial — flag, require admin to set date	Admin must set cutoff to generate stubs
OB_OFFSET posted but member fund balance goes negative	No	Always — reverse offset, re-classify cycle

Stub and instalment checks

Check	Auto-resolve?	Manual trigger?
UNRESOLVED stubs for a member with ACTIVE status	No	Always — clearance granted prematurely

Check	Auto-resolve?	Manual trigger?
Instalment collected but not linked to source stubs	Yes — link by member and cycle range	If ambiguous (multiple instalment plans)
Instalment total collected > total backdated obligation	Yes — refund excess to member cash	If excess > one instalment amount
ACTIVE member debited for a MIGRATION_WAIVED cycle	Yes — reverse debit	Never (always auto-reversed)
PARTIAL_CLEARANCE member accumulating delinquency on ring-fenced ESCALATED stubs	No	Always — exemption boundary violation

Migration ledger integrity assertion

```

ASSERT:  $\Sigma(\text{MIGRATION\_OPENING fund entries})$ 
      +  $\Sigma(\text{MIGRATION\_LUMPSUM or INSTALMENT collected})$ 
      +  $\Sigma(\text{MIGRATION\_OB\_OFFSET})$ 
      == expected total fund obligation for all migrated members

```

```

IF assertion fails:
    raise RECON_MIGRATION_LEDGER_DRIFT → manual queue

```

Manual exception queue

Exception record structure

Field	Description
exception_id	Unique, immutable
exception_type	Typed code (e.g. RECON_UNMATCHED_BANK_LINE)
domain	master_account · contribution · emi · loan · bank_clearing · late_fee · migration
severity	CRITICAL · HIGH · MEDIUM · LOW
amount_delta	Monetary discrepancy amount
affected_entities	member_id, cycle_id, loan_id, transaction_id
auto_resolve_attempted	Boolean + reason why auto-resolve was skipped or failed
raised_at	Detection timestamp
sla_deadline	Computed from severity + raised_at
assigned_to	Admin user — assigned on queue pickup
resolution	Action taken, journal entries posted, outcome
resolved_at	Admin resolution timestamp

Severity SLA

Severity	SLA	Batch behavior	Examples
CRITICAL	Immediate	Batch halted until resolved	MASTER_IMBALANCE_UNRESOLVED duplicate disbursement

Severity	SLA	Batch behavior	Examples
HIGH	Same business day	Affected member transactions held	Orphan credit, premature loan activation, unbanked cash >7d
MEDIUM	48 hours	Normal batch continues	Ambiguous bank match, wrong fee tier, amount mismatch
LOW	Next cycle	Normal batch continues	Stale pending entry, missing memo, minor timing diff

Admin resolution actions

Action	Description
Post correction entry	Admin selects corrective journal type; system validates it resolves the delta; posts with tag RECON_MANUAL_CORRECTION
Reverse transaction	Exact reversal of original entry with all legs mirrored; mandatory reason code required
Reclassify	Change transaction or cycle classification; old and new classification both logged permanently
Write-off	Post to designated write-off account; only available for LOW and MEDIUM severity
Escalate	Move to higher severity tier; SLA resets; mandatory escalation reason
Override and accept	Accept discrepancy without corrective entry; supervisor sign-off required

Algorithms

Nightly batch orchestrator

```

NIGHTLY_RECON_BATCH():
    snapshot = take_balance_snapshot(timestamp=now())

    // Step 1 - Master invariant (hard gate)
    result = check_master_invariant(snapshot)
    IF result == CRITICAL_FAIL:
        halt_all_posting()
        raise_exception(MASTER_IMBALANCE_UNRESOLVED, CRITICAL)
        RETURN // no further checks until resolved

    // Step 2 - Domain checks (parallel)
    run_in_parallel([
        reconcile_contributions(snapshot),
        reconcile_emi_and_loans(snapshot),
        reconcile_bank_clearing(snapshot),
        reconcile_late_fees(snapshot),
        reconcile_migration(snapshot)

```

```
])
```

```
// Step 3 - Auto-resolve where possible
FOR each exception raised:
  IF exception.auto_resolvable:
    attempt_auto_resolve(exception)
    IF auto_resolve_succeeds:
      log_auto_resolution(exception)
    ELSE:
      route_to_manual_queue(exception, severity=ESCALATED)
  ELSE:
    route_to_manual_queue(exception)

// Step 4 - Final invariant re-assertion
final_snapshot = take_balance_snapshot(timestamp=now())
re_check_master_invariant(final_snapshot)
write_recon_report(snapshot, final_snapshot)
```

Real-time event trigger

```
EVENT_TRIGGER: on_transaction_posted(txn):
  // Validate double-entry integrity
  IF txn.dr_total != txn.cr_total:
    raise_exception(UNBALANCED_ENTRY, CRITICAL)
    void_transaction(txn)
    RETURN

  // Validate account eligibility
  IF txn.member.status == 'MIGRATION_PENDING'
    AND txn.type NOT IN MIGRATION_ALLOWED_TYPES:
    raise_exception(INELIGIBLE_ACCOUNT_POSTING, HIGH)
    reverse_transaction(txn)
    RETURN

  // Validate fee exemptions
  IF txn.type == 'LATE_FEE'
    AND (txn.cycle.origin == 'MIGRATION'
        OR txn.member.has_active_loan
        OR txn.cycle.in_grace_period):
    auto_reverse_fee(txn)
    log_auto_resolution('RECON_AUTO_FEE_EXEMPTION_REVERSAL', txn)
    RETURN

  // Update running reconciliation state
  update_running_totals(txn)
  check_member_invariant(txn.member)
```

Member-level invariant check

```
FUNCTION check_member_invariant(member):
  expected_fund = member.opening_fund_balance
    +  $\Sigma$ (contributions collected)
    +  $\Sigma$ (migration instalments collected)
    -  $\Sigma$ (loan disbursements - member portion)
    +  $\Sigma$ (EMI repayments)
```

```

actual_fund    = member.fund_account.balance

IF abs(expected_fund - actual_fund) > tolerance:
    raise_exception(MEMBER_FUND_DRIFT, MEDIUM, member)

expected_cash = member.opening_cash_balance
               +  $\Sigma$ (deposits received)
               +  $\Sigma$ (loan disbursements credited)
               -  $\Sigma$ (contributions debited)
               -  $\Sigma$ (EMI debited)
               -  $\Sigma$ (late fees debited)
               -  $\Sigma$ (cash outs)
actual_cash    = member.cash_account.balance

IF abs(expected_cash - actual_cash) > tolerance:
    raise_exception(MEMBER_CASH_DRIFT, MEDIUM, member)

```